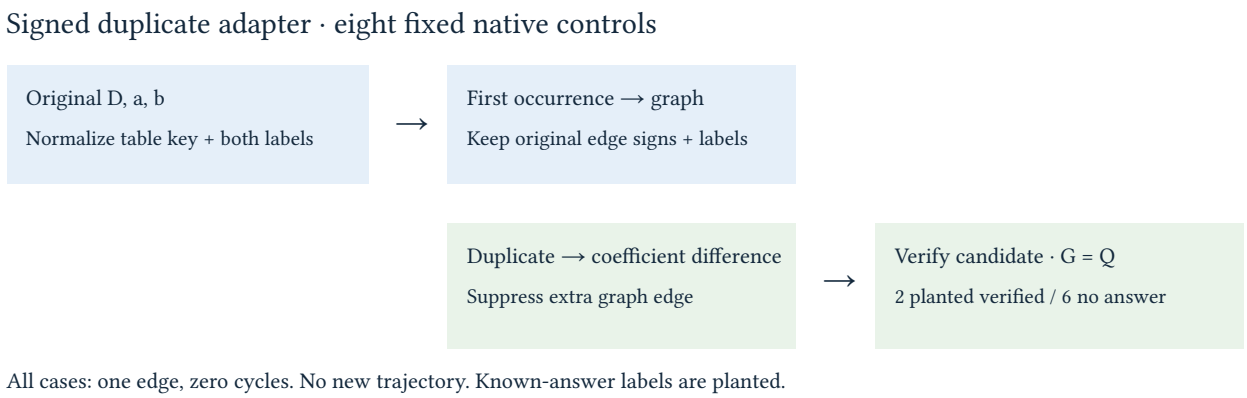
Native signed-duplicate label adapter



Eight frozen native controls passed with exit 0. The private helper retains the first canonically signed label pair, evaluates the coefficient difference before duplicate suppression, and verifies any returned candidate by multiplication in the original Jacobian. Both informative planted controls returned 31124. All eight graph snapshots remain unchanged after the duplicate: one edge, zero cycles.

The retained public fixture has p=739, r=34147, Q=31124G and D=2G+8Q with u=[38,223,1], v=[421,70]. Native checks establish Q=kG and the initial labelled divisor. The alternate informative pair (a+k,b-1) is deliberately constructed using the known answer. This checks the adapter interface on a retained geometric divisor; no new trajectory was executed and no naturally generated duplicate yield was measured.

The helper normalizes the divisor by min(D,-D) and applies that same sign to both labels. Its stored key is u=[38,223,1], v=[318,669], with initial labels [34145,34139]. The original graph insertion receives the ORIGINAL D and labels (2,8), yielding node signs [-1,1]. Canonicalization does not silently alter the first graph equation. The unchanged native graph adapter checks its factor sum, and before/after snapshots include equations, graph operations and scalar counts.

For two valid normalized occurrences, Delta_a+Delta_b*k=0 modulo r. A nonzero Delta_b gives candidate -Delta_a/Delta_b, which is returned only if candidate*G=Q. Zero Delta_b and nonzero Delta_a is an invariant error; both zero is redundant. The table is bound to one Jacobian, G and Q. Invalid relation labels are deliberately included as negative controls; the helper does not certify each incoming label invariant.

The constructor uses native Ring::new primality checking and requires odd prime r, r<=2^63, valid nonzero G, rG=0 and rQ=0. Prime r and nonzero G establish exact order r. Membership Q in <G> is a separate caller condition, demonstrated by Q=kG for this fixture; rQ=0 alone does not establish it in a general Jacobian. Canonical a,b<r are checked before arithmetic. The existing degree-two/distinct-root eligibility gate remains external.

Controls: informative same and opposite signs both verify 31124; redundant same and opposite signs produce zero equations; invalid zero-target coefficient returns invariant_error; invalid wrong candidate and changed-target input return candidate_rejected; a=r returns noncanonical_label. Changed-target control deliberately reuses invalid old labels with Q=2G and is not an example of legitimate cross-target pooling. No unexpected failures occurred; compiler dead-code warning is retained.

Added symbolic work: canonical divisor negation/comparison and table lookup; up to two Ring.neg calls per accepted input; two Ring.sub calls per duplicate; for a nonzero target coefficient, one additional negation, inversion and multiplication, then Jacobian scalar multiplication and equality verification. Native Ring.sub counts one add plus one negation, so its add counter increases twice per subtraction. Store two u64 labels per key, excluding container allocation and hashing overhead.

Observed second-input scalar counters (add,mul,inv) are (7,1,1) for informative same, (5,1,1) for informative opposite, (6,0,0)/(4,0,0) for redundant same/opposite, (6,0,0) for invariant_error, (7,1,1) for wrong-candidate and changed-target controls, and all zero for noncanonical rejection. Each informative second input records 22 Jacobian operations, 2422 M, 49 S, 77 I and 4188 field additions, including candidate verification. These are source instrumentation counts for fixed software fixtures, not timings or calibrated total costs.

Raw result.json retains initialization, first occurrence, graph processing, factor verification, incoming-label reference verification and second-input counters separately. Initial fixture validation has its own top-level counter. Fixture

construction, primality test implementation, hashing, comparisons, memory and general runtime work are not fully priced by those counters. Ring inversion records one inv rather than expanding its modular exponentiation. No performance ratio is inferred.

This is an isolated helper and bounded correctness harness, not a production collector integration. The existing graph deduplication behavior is preserved by invoking graph.consume only for the first occurrence. Next independent review should inspect helper assumptions, sign consistency, raw outcomes and source hashes, then replay in a private copy. Source precedent: Handbook of Applied Cryptography chapter 3, section 3.6.3 / Algorithm 3.60; retained signed-recovery work uses Chung–Costello–Smith. All original packets remain unchanged.